

UNIVERSIDADE FEDERAL DO TOCANTINS
CAMPUS UNIVERSITÁRIO DE PALMAS
CURSO DE CIÊNCIA DA COMPUTAÇÃO

Laboratório 5 – Barrel Shifter e Somador Carry Look-Ahead

Pedro Lucas Moreira

Relatório – Linguagens de Descrição de Hardware – 2026/2 – Setembro de
2026

Professor(a) Orientador(a): Tiago Almeida

The contents of this report are the sole responsibility of the authors.
O conteúdo deste relatório é de única responsabilidade dos autores.

Pedro Lucas Moreira

Laboratório 5 – Barrel Shifter e Somador Carry Look-Ahead

Relatório apresentado como parte das atividades da disciplina Linguagens de Descrição de Hardware – 2026/2, Curso de Ciência da Computação, Universidade Federal do Tocantins.

PALMAS / TO

2026

1 QUESTÃO 1 – BARREL SHIFTER DE 4 BITS

A rotação binária reposiciona os bits de uma palavra de forma circular, mantendo a ordem. Para a palavra $W = w_3w_2w_1w_0$ e a rotação $S = s_1s_0$, a saída $Y = y_3y_2y_1y_0$ segue a Tabela 1.

Tabela 1 – Tabela verdade do *barrel shifter* de 4 bits.

s_1	s_0	y_3	y_2	y_1	y_0
0	0	w_3	w_2	w_1	w_0
0	1	w_0	w_3	w_2	w_1
1	0	w_1	w_0	w_3	w_2
1	1	w_2	w_1	w_0	w_3

1.1 Item (a) – Implementação em VHDL

Arquivo a.vhdl:

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity rotation_binary is
5      port (
6          WORD_IN      : in  std_logic_vector(3 downto 0);
7          ROTATION_IN  : in  std_logic_vector(1 downto 0);
8          ROTATED_OUT  : out std_logic_vector(3 downto 0)
9      );
10 end entity rotation_binary;
11
12 architecture rtl of rotation_binary is
13 begin
14     with ROTATION_IN select
15         ROTATED_OUT(0) <= WORD_IN(0) when "00",
16                             WORD_IN(1) when "01",
17                             WORD_IN(2) when "10",
18                             WORD_IN(3) when others;
19
20     with ROTATION_IN select

```

```

21     ROTATED_OUT(1) <= WORD_IN(1) when "00",
22         WORD_IN(2) when "01",
23         WORD_IN(3) when "10",
24         WORD_IN(0) when others;
25
26     with ROTATION_IN select
27         ROTATED_OUT(2) <= WORD_IN(2) when "00",
28         WORD_IN(3) when "01",
29         WORD_IN(0) when "10",
30         WORD_IN(1) when others;
31
32     with ROTATION_IN select
33         ROTATED_OUT(3) <= WORD_IN(3) when "00",
34         WORD_IN(0) when "01",
35         WORD_IN(1) when "10",
36         WORD_IN(2) when others;
37 end architecture rtl;

```

1.1.1 Simulação

A simulação mantém uma palavra fixa em `WORD_IN` e percorre os quatro valores de `ROTATION_IN`, com 10 ns entre cada estímulo. São usadas duas palavras: 1000, com um único bit em 1 para deixar a rotação visível, e 1011.

Arquivo `tb_rotation_binary.vhdl`, usado para a simulação:

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity tb_rotation_binary is
6  end entity tb_rotation_binary;
7
8  architecture sim of tb_rotation_binary is
9      signal WORD_IN      : std_logic_vector(3 downto 0);
10     signal ROTATION_IN  : std_logic_vector(1 downto 0);
11     signal ROTATED_OUT  : std_logic_vector(3 downto 0);
12 begin
13     uut: entity work.rotation_binary
14         port map (

```

```

15     WORD_IN      => WORD_IN,
16     ROTATION_IN => ROTATION_IN,
17     ROTATED_OUT => ROTATED_OUT
18 );
19
20 stim: process
21 begin
22     — palavra fixa com bits distintos para visualizar a rotacao
23     WORD_IN <= "1000";
24     for s in 0 to 3 loop
25         ROTATION_IN <= std_logic_vector(to_unsigned(s, 2));
26         wait for 10 ns;
27     end loop;
28
29     WORD_IN <= "1011";
30     for s in 0 to 3 loop
31         ROTATION_IN <= std_logic_vector(to_unsigned(s, 2));
32         wait for 10 ns;
33     end loop;
34     wait;
35 end process;
36 end architecture sim;

```

Resultados esperados, obtidos a partir da Tabela 1:

Tabela 2 – Saídas esperadas na simulação.

WORD_IN	ROTATION_IN	ROTATED_OUT
1000	00	1000
1000	01	0100
1000	10	0010
1000	11	0001
1011	00	1011
1011	01	1101
1011	10	1110
1011	11	0111

1.2 Item (b) – Execução na DE1 com o demo_setup

O `rotation_binary` foi instanciado no `demo_setup`. Os *switches* SW(3 downto 0) fornecem a palavra W , SW(5 downto 4) fornecem o rotacionamento S , e a saída Y é exibida nos LEDs vermelhos LEDR3 a LEDR0.

Arquivo `b.vhdl`:

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity demo_setup is
5      port (
6          SW      : in  std_logic_vector(5 downto 0);
7          LEDR0   : out std_logic;
8          LEDR1   : out std_logic;
9          LEDR2   : out std_logic;
10         LEDR3   : out std_logic
11     );
12 end entity demo_setup;
13
14 architecture rtl of demo_setup is
15     signal rotated : std_logic_vector(3 downto 0);
16 begin
17     shifter_inst: entity work.rotation_binary
18         port map (
19         WORD_IN      => SW(3 downto 0),
20         ROTATION_IN => SW(5 downto 4),
21         ROTATED_OUT => rotated
22     );
23
24     LEDR0 <= rotated(0);
25     LEDR1 <= rotated(1);
26     LEDR2 <= rotated(2);
27     LEDR3 <= rotated(3);
28
29 end architecture rtl;

```

2 QUESTÃO 2 – SOMADOR CARRY LOOK-AHEAD

2.1 Item (a) – CLA de 4 bits

O somador *carry look-ahead* (CLA) calcula todos os carries em paralelo, em vez de esperar o carry atravessar cada estágio como no *ripple-carry*.

Arquivo a.vhdl:

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity cla_adder_4bit is
5      port (
6          A    : in  std_logic_vector(3 downto 0);
7          B    : in  std_logic_vector(3 downto 0);
8          Cin   : in  std_logic;
9          S    : out std_logic_vector(3 downto 0);
10         Cout  : out std_logic
11     );
12 end entity cla_adder_4bit;
13
14 architecture rtl of cla_adder_4bit is
15     signal G : std_logic_vector(3 downto 0);
16     signal P : std_logic_vector(3 downto 0);
17     signal C : std_logic_vector(4 downto 0);
18 begin
19     process (A, B)
20     begin
21         for i in 0 to 3 loop
22             G(i) <= A(i) and B(i);
23             P(i) <= A(i) xor B(i);
24         end loop;
25     end process;
26
27     C(0) <= Cin;
28
29     C(1) <= G(0) or (P(0) and C(0));
30

```

```

31 C(2) <= G(1)
32     or (P(1) and G(0))
33     or (P(1) and P(0) and C(0));
34
35 C(3) <= G(2)
36     or (P(2) and G(1))
37     or (P(2) and P(1) and G(0))
38     or (P(2) and P(1) and P(0) and C(0));
39
40 C(4) <= G(3)
41     or (P(3) and G(2))
42     or (P(3) and P(2) and G(1))
43     or (P(3) and P(2) and P(1) and G(0))
44     or (P(3) and P(2) and P(1) and P(0) and C(0));
45
46 process (P, C)
47 begin
48     for i in 0 to 3 loop
49         S(i) <= P(i) xor C(i);
50     end loop;
51 end process;
52
53 Cout <= C(4);
54
55 end architecture rtl;

```

2.1.1 Simulação

A simulação percorre as 512 combinações de A, B e Cin, com 10 ns entre cada estímulo. O resultado foi **0 erros** nas 512 combinações.

Arquivo `tb_cla_adder_4bit.vhdl`, usado para a simulação:

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 — Testbench exaustivo: percorre as 512 combinacoes de A, B e Cin
6 — e compara a saida do CLA com a soma calculada via numeric_std.
7 entity tb_cla_adder_4bit is

```

[illegible]

```

47         severity error;
48     end if;
49 end loop;
50 end loop;
51 end loop;
52
53 report "Fim da simulacao: " & integer'image(errors) & " erro(s)"
54     severity note;
55 wait;
56 end process;
57 end architecture sim;

```

2.1.2 Comparação de tempo com o ripple-carry de 4 bits

O CLA e o ripple_carry_adder4 do laboratório anterior foram compilados nas mesmas condições.

Tabela 3 – Caminho crítico dos somadores de 4 bits.

Somador	Caminho crítico	Atraso	LEs
CLA 4 bits	B[2] → Cout	13,129 ns	15
Ripple-carry 4 bits	y[2] → overflow	12,475 ns	10

Em 4 bits, o *ripple-carry* teve o menor tempo, 0,654 ns (cerca de 5%) a menos que o CLA.

A vantagem do CLA aparece quando o número de bits cresce: no *ripple-carry* o atraso aumenta com N (no laboratório anterior, 19,860 ns em 8 bits e 71,831 ns em 64 bits), enquanto no CLA o carry não depende da cadeia de estágios anteriores.

2.1.3 Execução na DE1 com o demo_setup

O cla_adder_4bit foi instanciado no demo_setup, com os *switches* como entradas e os LEDs vermelhos como saídas.

Arquivo demo_setup.vhd:

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity demo_setup is

```

Tabela 4 – Mapeamento de entradas e saídas na DE1 (CLA).

Sinal	Função	Pino
SW[3..0]	$A_3..A_0$	V12, M22, L21, L22
SW[7..4]	$B_3..B_0$	M2, U11, U12, W12
SW[8]	C_{in}	M1
LEDR3..LEDR0	$S_3..S_0$	Y19, U19, R19, R20
LEDR4	C_{out}	T18

```

5  port (
6      SW      : in  std_logic_vector(8 downto 0);
7      LEDR0   : out std_logic;
8      LEDR1   : out std_logic;
9      LEDR2   : out std_logic;
10     LEDR3   : out std_logic;
11     LEDR4   : out std_logic
12 );
13 end entity demo_setup;
14
15 architecture rtl of demo_setup is
16     signal sum_result : std_logic_vector(3 downto 0);
17     signal carry_out  : std_logic;
18 begin
19     adder_inst: entity work.cla_adder_4bit
20         port map (
21             A    => SW(3 downto 0) ,
22             B    => SW(7 downto 4) ,
23             Cin  => SW(8) ,
24             S    => sum_result ,
25             Cout => carry_out
26         );
27
28     LEDR0 <= sum_result(0);
29     LEDR1 <= sum_result(1);
30     LEDR2 <= sum_result(2);
31     LEDR3 <= sum_result(3);
32     LEDR4 <= carry_out;
33
34 end architecture rtl;

```

2.2 Item (b) – CLA parcial de 8 bits

O somador de 8 bits com CLA parcial é formado por dois blocos CLA de 4 bits ligados em cascata: o carry-out do bloco menos significativo, C_4 , é o carry-in do bloco mais significativo. Dentro de cada bloco os carries são calculados em paralelo; o único elo em série é C_4 . A saída Cout_half_1 expõe o carry entre os blocos (C_4) e Cout_half_2 é o carry-out final (C_8).

Arquivo b.vhdl:

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity cla_adder_8bit_with_4bit is
5      port (
6          A          : in  std_logic_vector(7 downto 0);
7          B          : in  std_logic_vector(7 downto 0);
8          Cin        : in  std_logic;
9          S          : out std_logic_vector(7 downto 0);
10         Cout_half_1 : out std_logic;
11         Cout_half_2 : out std_logic
12     );
13 end entity cla_adder_8bit_with_4bit;
14
15 architecture rtl of cla_adder_8bit_with_4bit is
16     signal G : std_logic_vector(7 downto 0);
17     signal P : std_logic_vector(7 downto 0);
18     signal C : std_logic_vector(8 downto 0);
19 begin
20     gp_low: for i in 0 to 3 generate
21         G(i) <= A(i) and B(i);
22         P(i) <= A(i) xor B(i);
23     end generate gp_low;
24
25     C(0) <= Cin;
26
27     C(1) <= G(0) or (P(0) and C(0));
28
29     C(2) <= G(1)
30         or (P(1) and G(0))

```

```

31         or (P(1) and P(0) and C(0));
32
33     C(3) <= G(2)
34         or (P(2) and G(1))
35         or (P(2) and P(1) and G(0))
36         or (P(2) and P(1) and P(0) and C(0));
37
38     C(4) <= G(3)
39         or (P(3) and G(2))
40         or (P(3) and P(2) and G(1))
41         or (P(3) and P(2) and P(1) and G(0))
42         or (P(3) and P(2) and P(1) and P(0) and C(0));
43
44     sum_low: for i in 0 to 3 generate
45         S(i) <= P(i) xor C(i);
46     end generate sum_low;
47
48     Cout_half_1 <= C(4);
49
50     gp_high: for i in 4 to 7 generate
51         G(i) <= A(i) and B(i);
52         P(i) <= A(i) xor B(i);
53     end generate gp_high;
54
55     C(5) <= G(4) or (P(4) and C(4));
56
57     C(6) <= G(5)
58         or (P(5) and G(4))
59         or (P(5) and P(4) and C(4));
60
61     C(7) <= G(6)
62         or (P(6) and G(5))
63         or (P(6) and P(5) and G(4))
64         or (P(6) and P(5) and P(4) and C(4));
65
66     C(8) <= G(7)
67         or (P(7) and G(6))
68         or (P(7) and P(6) and G(5))
69         or (P(7) and P(6) and P(5) and G(4))

```

```

70         or (P(7) and P(6) and P(5) and P(4) and C(4));
71
72     sum_high: for i in 4 to 7 generate
73         S(i) <= P(i) xor C(i);
74     end generate sum_high;
75
76     Cout_half_2 <= C(8);
77 end architecture rtl;

```

2.3 Item (c) – CLA puro de 8 bits

Todos os carries até C_8 são expandidos diretamente em função de G , P e C_{in} , sem cascata entre blocos.

Arquivo c.vhdl:

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity cla_adder_8bit is
5      port (
6          A    : in  std_logic_vector(7 downto 0);
7          B    : in  std_logic_vector(7 downto 0);
8          Cin   : in  std_logic;
9          S     : out std_logic_vector(7 downto 0);
10         Cout  : out std_logic
11     );
12 end entity cla_adder_8bit;
13
14 architecture rtl of cla_adder_8bit is
15     signal G : std_logic_vector(7 downto 0);
16     signal P : std_logic_vector(7 downto 0);
17     signal C : std_logic_vector(8 downto 0);
18 begin
19     process (A, B)
20     begin
21         for i in 0 to 7 loop
22             G(i) <= A(i) and B(i);
23             P(i) <= A(i) xor B(i);
24         end loop;

```

```

25  end process ;
26
27  C(0) <= Cin ;
28
29  C(1) <= G(0) or (P(0) and C(0)) ;
30
31  C(2) <= G(1)
32      or (P(1) and G(0))
33      or (P(1) and P(0) and C(0)) ;
34
35  C(3) <= G(2)
36      or (P(2) and G(1))
37      or (P(2) and P(1) and G(0))
38      or (P(2) and P(1) and P(0) and C(0)) ;
39
40  C(4) <= G(3)
41      or (P(3) and G(2))
42      or (P(3) and P(2) and G(1))
43      or (P(3) and P(2) and P(1) and G(0))
44      or (P(3) and P(2) and P(1) and P(0) and C(0)) ;
45
46  C(5) <= G(4)
47      or (P(4) and G(3))
48      or (P(4) and P(3) and G(2))
49      or (P(4) and P(3) and P(2) and G(1))
50      or (P(4) and P(3) and P(2) and P(1) and G(0))
51      or (P(4) and P(3) and P(2) and P(1) and P(0) and C(0)) ;
52
53  C(6) <= G(5)
54      or (P(5) and G(4))
55      or (P(5) and P(4) and G(3))
56      or (P(5) and P(4) and P(3) and G(2))
57      or (P(5) and P(4) and P(3) and P(2) and G(1))
58      or (P(5) and P(4) and P(3) and P(2) and P(1) and G(0))
59      or (P(5) and P(4) and P(3) and P(2) and P(1) and P(0) and C(0)) ;
60
61  C(7) <= G(6)
62      or (P(6) and G(5))
63      or (P(6) and P(5) and G(4))

```

```

64      or (P(6) and P(5) and P(4) and G(3))
65      or (P(6) and P(5) and P(4) and P(3) and G(2))
66      or (P(6) and P(5) and P(4) and P(3) and P(2) and G(1))
67      or (P(6) and P(5) and P(4) and P(3) and P(2) and P(1) and G(0))
68      or (P(6) and P(5) and P(4) and P(3) and P(2) and P(1) and P(0)
        and C(0));
69
70      C(8) <= G(7)
71      or (P(7) and G(6))
72      or (P(7) and P(6) and G(5))
73      or (P(7) and P(6) and P(5) and G(4))
74      or (P(7) and P(6) and P(5) and P(4) and G(3))
75      or (P(7) and P(6) and P(5) and P(4) and P(3) and G(2))
76      or (P(7) and P(6) and P(5) and P(4) and P(3) and P(2) and G(1))
77      or (P(7) and P(6) and P(5) and P(4) and P(3) and P(2) and P(1)
        and G(0))
78      or (P(7) and P(6) and P(5) and P(4) and P(3) and P(2) and P(1)
        and P(0) and C(0));
79
80      process (P, C)
81      begin
82          for i in 0 to 7 loop
83              S(i) <= P(i) xor C(i);
84          end loop;
85      end process;
86
87      Cout <= C(8);
88
89      end architecture rtl;

```

Os itens (b) e (c) foram verificados com *testbench* exaustivo (131.072 combinações de A, B e Cin): 0 erros.

2.4 Análise de tempos dos somadores de 8 bits

Os três somadores de 8 bits foram compilados nas mesmas condições do item (a) e medidos com o TimeQuest. O *ripple-carry* de 8 bits do laboratório anterior foi recompilado e resultou em 19,860 ns, exatamente o valor medido naquele laboratório, o que confirma que as condições de medição são as mesmas.

Tabela 5 – Caminho crítico dos somadores de 8 bits.

Somador	Caminho crítico	Atraso	LEs
Ripple-carry (lab. 4)	x[0] → overflow	19,860 ns	18
CLA parcial (b)	A[0] → Cout_half_2	16,087 ns	34
CLA puro (c)	B[2] → S[6]	15,880 ns	42

O menor tempo é o do **CLA puro**. Diferente do resultado em 4 bits, em 8 bits as duas versões de CLA são mais rápidas que o *ripple-carry*:

- **CLA parcial × ripple-carry:** 3,773 ns a menos (cerca de 19%). No *ripple-carry* o carry precisa atravessar 8 somadores completos em série; no CLA parcial ele só passa em série pelo elo C_4 entre os dois blocos. O caminho crítico ainda vai do bit 0 até o carry final, atravessando essa cascata.
- **CLA puro × CLA parcial:** 0,207 ns a menos (cerca de 1%). Como nenhum carry depende de outro carry, o elo C_4 deixa de existir, e o caminho crítico passa a terminar em um bit da soma (S[6]) e não mais no carry-out. O ganho é pequeno porque a síntese já reorganiza as duas descrições em LUTs de 4 entradas, e C_8 , com 17 entradas, precisa de vários níveis de LUT em qualquer das duas formas.
- **CLA puro × ripple-carry:** 3,980 ns a menos (cerca de 20%).